What the repo genuinely does well:

The mission clarity in CLAUDE.md is exceptional. The invariants ("never reintroduce placeholder zero-vector behavior", "search results must refer to actual stored observations") show architectural discipline. The 5-layer separation of Capture → Observation → World-Model → Proof → Extension is a genuinely good abstraction that would survive a major engine swap. The multi-platform SDK strategy (Python, TypeScript, Swift, Kotlin) means the proof surface could reach real consumer devices without a rewrite. The "baseline comparison" feature (JEPA vs caption-only vs retrieval-only on the same scene) is the most underrated design decision in the whole system — it's the thing that will make JEPA legible to non-researchers.

Where it actually fails as a JEPA system (my honest, deeper critique beyond what the review document said):

The review document identified the Generative Trap and Storage-based state. Those are real. But four deeper problems weren't named:

Gap 3 — No predictor network exists at all. JEPA's mathematical core is a learned parametric function `f_θ(s_context, m)` that maps a context representation and a mask token into a predicted target representation. The "PredictionWindow" in Toori's world model is not this. It is a statistical bookkeeping structure over stored embeddings, not a forward pass through a trained neural predictor. There is no θ. There is no training signal. This means the "prediction" Toori currently claims to make is not a prediction in any ML sense — it is a nearest-neighbor lookup dressed in prediction language.

Gap 4 — No target encoder / EMA mechanism. JEPA requires two encoders: a context encoder that sees unmasked patches, and a target encoder (an exponential moving average of the context encoder) that produces stable, non-collapsing targets. Without the EMA target encoder, any online learning loop would undergo representation collapse — all embeddings would converge to a single point. The current ONNX-based perception is a frozen pre-trained model, not an EMA-updated target encoder. This is fine as a starting point but it means the system cannot learn from the specific scenes it observes.

Gap 5 — The "surprise" metric is not energy-based. True VoE surprise in a JEPA world model is the scalar `E = ||s_context - ŝ_predicted||²` computed entirely in embedding space. What Toori currently computes is likely either (a) cosine distance between consecutive frame embeddings, which measures appearance change, not prediction violation, or (b) an LLM's verbal assessment of change. Neither of these is JEPA surprise. The first measures "how different does this look" rather than "how much did the model's prediction fail." The second collapses continuous physics into language tokens. The distinction matters because a scene can look identical between frames while the world model's prediction fails spectacularly (e.g., a ball that was expected to continue rolling but stopped), and vice versa.

Gap 6 — Temporal continuity is database-centric, not dynamics-centric. In a proper video JEPA, temporal continuity is maintained by the predictor's learned dynamics: given `s_t`, the predictor produces `ŝ_{t+1}`, and the actual encoder output `s_{t+1}` is then

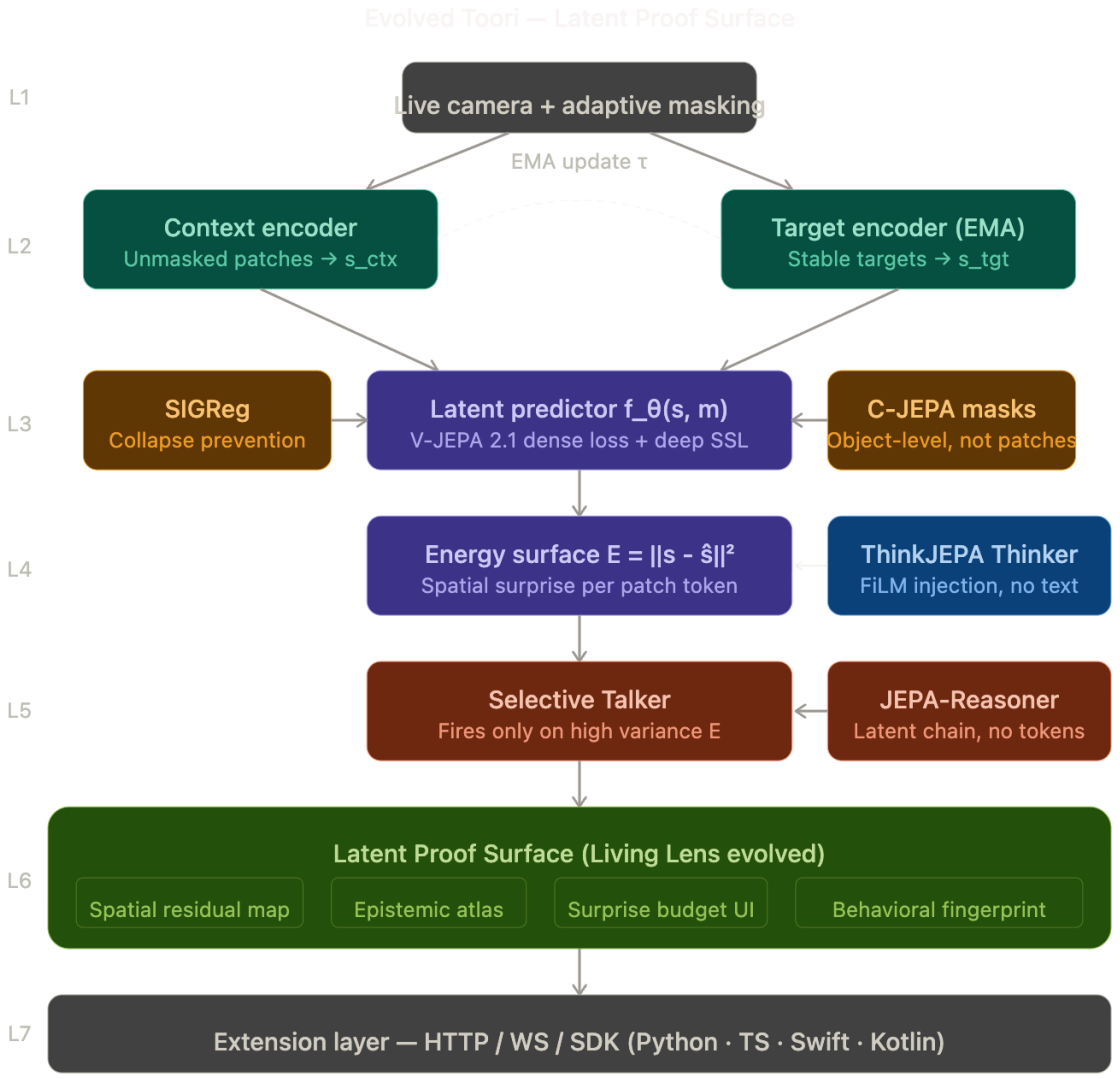
compared to `ŝ_{t+1}`. The residual is the signal. Toori instead manages continuity by querying a SQLite database for prior observations and computing similarity. This is not temporal dynamics — it is temporal retrieval. The difference: a retrieval system can only say "this looks like what I saw before." A dynamics system says "this is what I expected to see, and here is exactly how reality deviated from my model."

Now let me visualize the architectural gap clearly before proposing the evolution path:

Part II — My Quintessential Innovation: The "Latent Proof Surface" Architecture

The prior review gave you good component-level advice (swap in V-JEPA 2.1, add ThinkJEPA). My deeper contribution is a different idea: **Toori's unique value is not the JEPA engine itself — it's the proof surface that makes the engine's internal state inspectable.** No one else is building this. Meta, Google, and others are building JEPA engines; nobody is building the UI layer that makes those engines legible to operators, product teams, and ultimately consumers. That is the market-dominating wedge.

So my proposal is not just "upgrade the engine." It is: **build the world's first Latent State Inspector**, where every abstraction level of the JEPA computation — the mask, the context embedding, the predicted embedding, the target embedding, the energy residual — is rendered visually in real time, mapped back onto the original image as a spatial overlay. This is what "JEPA proof surface" actually means at its most ambitious. Here is the complete evolved architecture:



Part III — My Five Original Innovation Ideas (beyond the research papers)

The papers give you ingredients. These are recipes no one has built yet:

1. The Spatial Residual Canvas. Overlay the energy map `E(patch_i) = ||s_i - ŝ_i||²` directly onto the live camera feed as a real-time heatmap. Cold patches (low prediction error) stay neutral. Hot patches (high surprise) glow amber/red. This makes the JEPA predictor's mental model literally visible. No LLM in the loop, no text output — just pure geometry. When you occlude an object with your hand, the model's residual flares exactly where it expected the object to be. This is the single most compelling demo you can build, and it runs entirely in embedding space.

2. The Epistemic Atlas. Replace the flat SQLite observation log with a scene-level entity graph. Nodes are latent entity centroids (tracked by C-JEPA's object-level masking). Edges are the predicted interaction energies between entities. The graph persists across sessions. When an entity disappears and reappears, the node re-connects to its historical centroid via a dashed edge. This graph is your "world model state" made visible — not a list of captions, but a topological memory of which things have existed in which relationships. It answers "what does Toori know?" in a form that is genuinely inspectable.

3. The Surprise Budget Controller. A dynamic masking curriculum: allocate more mask tokens to the regions of the image where the predictor's recent prediction error was highest. This is an online active learning strategy — the system automatically focuses its predictive capacity on the parts of the scene where its world model is weakest. Practically: divide the frame into a grid, track rolling mean prediction error per cell, sample the masking distribution proportionally. This makes the system get smarter about the specific environment it is watching, not just the generic distribution it was trained on.

4. The Behavioral Fingerprint. Compress each session's temporal dynamics into a single compact vector: the mean and variance of the energy surface over time, combined with the dominant eigenvectors of the entity interaction graph. Two sessions in the same room will have similar fingerprints. Two sessions with unusual events will have high fingerprint distance from the baseline. This unlocks cross-session anomaly detection without storing any raw frames or running any LLM. It is a privacy-preserving ambient intelligence layer.

5. The Structured Language Bridge (the practical compromise that makes this shippable). Do not eliminate language output entirely — eliminate *uncontrolled* language output. Instead of asking Ollama "describe what changed," define a small closed vocabulary of latent events (ENTITY_APPEARED, ENTITY_DISAPPEARED, OCCLUSION_START, OCCLUSION_END, PREDICTION_VIOLATION, SCENE_STABLE) and have the selective Talker only classify the current latent state into one of these categories. This is a 6-class classifier, not a language model, and it can run in under 10ms on-device. The UI then renders these structured events with rich spatial overlays rather than generated text. This gives you inspectable, fast, non-hallucinating language-level output from a pure latent computation.

Part IV — Current vs Future System Comparison

Dimension	Current Toori	Evolved Toori

Core computation	Frozen ONNX → SQLite → Ollama text	Context encoder + EMA target encoder + learned f_θ predictor
Surprise metric	Cosine distance / LLM verbal assessment	Spatial energy E =
Temporal continuity	Database retrieval + similarity lookup	Latent dynamics forward pass
Object tracking	Global embedding similarity	C-JEPA object-level masking, causal interaction
Language output	Every-frame Ollama generation	Selective Talker on high-variance E only
UI output	Text captions + metadata	Spatial residual heatmap + entity graph + structured events
Training	No training loop	Online SIGReg-stabilized SSL with EMA update
Semantic guidance	Ollama reasoning over full scene text	ThinkJEPA FiLM injection into predictor, no text
Masking strategy	None	C-JEPA object-level + Surprise Budget curriculum
Edge deployment	Constrained by Ollama latency	Selective Talker cuts decode ops ~2.85x

---